Home: https://www.postgresql.org/docs/current/plpgsql.html

PLPGSQL
PL = **procedural language** for PostgreSQL.

e.g.

```
DO LANGUAGE PLPGSQL $$
BEGIN
  RAISE INFO 'hello'
END;
$$;
```

e.g. FOR loop

```
DO LANGUAGE PLPGSQL $$
DECLARE
  name text;
BEGIN
  FOR name IN SELECT nspname FROM pg_namespace
  LOOP
    RAISE INFO 'namespace: %', name;
  END LOOP;
END;
$$;
```

*** configurations**
https://www.postgresql.org/docs/12/plpgsql-development-tips.html

plpgsql.check_asserts='on' boolean
plpgsql.extra_warnings='none' none | all | <opt>[,...]
plpgsql.extra_errors='none' none | all | <opt>[,...]
where opts
shadowed_variables

too_many_rows ... INTO x received more than 1 row

strict_multi_assignment

plpgsql.print_strict_params='off' boolean
include error details in STRICT

plpgsql.variable_conflict='error' error|use_variable|use_column
how to handle variable and column
name conflict

*** usage: OUT parameter assignment**
OUT parameters can be assigned to **variables** in a PLPGSQL code.

e.g.
Step 1) Define procedure

```
-- Step 1) define procedure
```

```
CREATE OR REPLACE PROCEDURE f(
  a int,
  OUT x int,                  -- NULL initially
  OUT y text,                 -- NULL
  OUT z boolean               -- NULL
) as $$
BEGIN
    x := a + 1;               -- set x
    y := 'hello';             -- set y
END;
$$ LANGUAGE PLPGSQL;
```

Step 2)

```
-- usage 1: call in SQL
> CALL f(1, NULL, NULL, NULL);
> CALL f(1, 1, 'a', TRUE);             -- same. initial
values for OUT parameters are ignored

 x |   y   |  z
---+-------+------
 2 | hello | NULL


-- usage 2: call in PLPGSQL
> DO LANGUAGE PLPGSQL $$
DECLARE
  x int;
  y text;
  z boolean;
BEGIN
  CALL f(100, x, y, z);           -- sets x=101
y='hello' z=NULL
  RAISE INFO '% % %', x, y, z;
END;
$$;

INFO:  101 hello <NULL>
```

==========
======

== SQL execution ==

Ref: https://www.postgresql.org/docs/current/plpgsql-statements.html

In general, any SQL can be executed **directly**.
e.g.

```
-- direct
SELECT 1 INTO x;
INSERT INTO t VALUES (1);
```

```
-- dynamic
EXECUTE 'SELECT 1' INTO x;
```

However, **features** differ depending on the SQL **type**.
2 types:
1) **cacheable** SQL
Query plan is **cached**
Same as **PREPARE**d statements.

Ref: https://www.postgresql.org/docs/current/sql-prepare.html
SELECT ...
INSERT ...
UPDATE ...
DELETE ...
MERGE ...

2) **non-cacheable** SQL
Everything else.
e.g.
CREATE TABLE ...
COPY ...

*** Feature: execution**
SQL result must either be **assigned** to a variable or explicitly **discarded**.

case A: SQL **queries**

```
-- case: SQL only
-- FORBIDDEN since the output is unhandled
SELECT 1;                       -- ERROR


-- usage 1: PERFORM
-- i.e. discard output
PERFORM 1;                      -- OK  Same as 'SELECT 1'
PERFORM * FROM t;               -- OK

-- usage 2: RETURN
RETURN QUERY SELECT 1;          -- OK


-- usage 3: assign single output to variable
-- LOOP if multiple.
SELECT 1 INTO x;                    -- OK
SELECT * FROM pg_class INTO x;      -- ERROR  Must be
exactly ONE row
                                    -- Use loop instead
FOR x IN                            -- OK LOOP
   SELECT * FROM pg_class
LOOP
   ... x.relid ...
END LOOP;
```

case B: SQL **statements**

```
-- usage 1: call as usual
INSERT INTO t VALUES (1);              -- OK: no output
by default

-- usage 2: assign RETURNING
INSERT INTO t VALUES (1)               -- OK
  RETURNING t.value INTO x;

INSERT INTO t VALUES (1), (2)          -- ERROR Must be
exactly one row
  RETURNING t.value INTO x;            -- Use LOOP
instead

FOR x IN                               -- OK: LOOP
  INSERT INTO t VALUES (1), (2)
    RETURNING t.value
LOOP
  ...
END LOOP;
```

*** Feature: parameter/variable**

Only **cacheable queries** can use parameters directly
e.g.

```
CREATE PROCEDURE f(value INT) AS $$
DECLARE
  x INT;
BEGIN
  SELECT value INTO
x;                                 -- OK: can use
variable in cacheable SQL

  CREATE TABLE mytable (x INT DEFAULT
1);              -- OK: can execute without variable

  CREATE TABLE mytable (x INT DEFAULT
value);          -- RUNTIME ERROR: can't use variable in
non-cacheable SQL

  EXECUTE 'CREATE TABLE mytable (x INT DEFAULT %L)'
    -- RUNTIME ERROR: can't use parameter either
    USING value;

  EXECUTE
format(                                          -- OK:
dynamic execution
    'CREATE TABLE mytable (x INT DEFAULT %L)',
    value
   );
```

```
END;
$$ LANGUAGE PLPGSQL;
```

IMPORTANT: DON'T BUILD DYNAMIC QUERIES MANUALLY
May easily lead to SQL injection

```
EXECUTE 'SELECT * FROM '                    -- NEVER
  || tablename
  || 'WHERE x ='
  || value;

EXECUTE 'SELECT * FROM '                    -- OK, but
unsafe
  || quote_ident(tablename)
  || 'WHERE x ='
  || quote_literal(value);

EXECUTE format(                             -- GOOD
  'SELECT * FROM %I WHERE x=$1,             -- %I   same
as quote_ident
  tablename
)
  USING value;
```

*** usage: execution error handling**

e.g.

```
BEGIN
  -- case: non-strict
  --
  -- if more than 1 row:
  SELECT ... INTO x;              -- may raise
TOO_MANY_ROW
  IF NOT FOUND THEN               -- check for
  END IF

  SELECT ... INTO STRICT x;       -- may raise
TOO_MANY_ROWS

EXCEPTION
  WHEN NO_DATA_FOUND THEN
    ...
  WHEN TOO_MANY_ROWS THEN
    ....

-- Error handling

IF NOT FOUND THEN
  ....
```

```
END IF
EXCEPTION
  WHEN TOO_MANY_ROWS THEN
    ....
```

*** Execution checks (FOUND, DIAGNOSTICS)**
https://www.postgresql.org/docs/current/plpgsql-statements.html

FOUND variable
Special variable indicating whether a query had **at least ONE** result.
e.g.

```
VALUES (1) INTO x;                          -- 1 row
FOUND = TRUE

SELECT * FROM t WHERE FALSE INTO x;         -- 0 rows
FOUND = FALSE

VALUES (1), (2) INTO x;                     -- >1 rows
FOUND = TRUE

-- with STRICT
SELECT ... INTO STRICT x;                   -- 0 rows
NO_DATA_FOUND error
                                            -- >1 rows
TOO_MANY_ROWS error
```

Action	**FOUND**
SELECT ... INTO x	FALSE if 0 rows
PERFORM ...	FALSE if 0 rows
UPDATE ... INTO x INSERT ... INTO x DELETE ... INTO x MERGE ... INTO x	FALSE if 0 rows changed
RETURN QUERY ...	FALSE if 0 rows
FOR ... LOOP	FALSE if 0 loop
EXECUTE ...	NA use GET DIAGNOSTICS

```
-- FOUND
SELECT ... INTO x;
IF NOT FOUND THEN                       -- FOUND=TRUE if x
was set
  RAISE EXCEPTION 'x not found';
END IF;
```

```
-- GOTCHA: FOUND not supported for EXECUTE
EXECUTE ...;
GET DIAGNOSTICS row_count := ROW_COUNT;
IF row_count = 0 THEN
  ...
END IF;


-- DIAGNOSTICS: meta info about routine
-- e.g. stacktrace
--
c BIGINT;
ctx TEXT;
id  oid;
GET DIAGNOSTICS
  c := ROW_COUNT,             -- number of rows returned
by previous query
                              -- e.g. 1
  ctx := PG_CONTEXT,          -- current stacktrace
                              -- e.g. "PL/pgSQL function
f() line 2 at GET DIAGNOSTICS"
  id := PG_ROUTINE_OID;       -- e.g. 12345
```

*** Syntax**

Ref: https://www.postgresql.org/docs/current/plpgsql-statements.html

```
-- EXECUTE: dynamic SQL
--
-- IMPORTANT: dynamic SQL isn't cached
EXECUTE 'INSERT INTO mytable VALUES (1, 2)';
EXECUTE 'INSERT INTO mytable VALUES ($1, $2)'      --
parameters
  USING x, y;
EXECUTE 'SELECT 1' INTO x;                         --
assignment

-- GOTCHA: can't use parameters for identifiers
EXECUTE 'INSERT INTO $1 VALUES (1, 2)';
    -- ERROR
  USING tablename;
EXECUTE format('INSERT INTO %I VALUES (1, 2)',
tablename);  -- OK


-- PERFORM: discard result
PERFORM 1;                        -- OK    Discard
return
                                  --       Replace
'SELECT' with 'PERFORM'
PERFORM pg_reload_conf();         -- e.g. typically for
side-effect
```

```
-- INTO assignment
--
-- a) SELECT ... INTO
SELECT n INTO x FROM mytable;              -- x=NULL if
empty
                                           -- x=first
row if multiple
SELECT array_agg(n) INTO x FROM mytable;   -- OK
SELECT 1, 'a' INTO r;                      -- record
variable
                                           --  r RECORD
where r.column1=1  r.column2='a'
SELECT 1, 'a' INTO x, y;                   -- multiple
variables
                                           -- x=1 y='a'


-- b) RETURNING ... INTO
INSERT INTO mytable
  VALUES (100, 'a')
  RETURNING mytable.id
  INTO x;                                  -- x = 100
                                           -- x=NULL if
empty
                                           -- x=first
row if multiple

UPDATE ... RETURNING mytable.id INTO x;
DELETE ... RETURNING mytable.id INTO x;
MERGE ... RETURNING mytable.id INTO x;

-- composite type
INSERT INTO mytable               -- d mytable  i.e. d
is of type "mytable"
  ...
  RETURNING (mytable).*
  RETURNING (mytable)             -- BAD
  INTO d;


-- STRICT: MUST return exactly ONE row.
-- In particular, no return value is also an error
SELECT ... INTO STRICT x;                  --
TOO_MANY_ROWS error if more than 1 row
                                           --
NO_DATA_FOUND error if NULL
INSERT ... RETURNING t INTO STRICT x;      -- same with
INSERT ... RETURNING
UPDATE ... RETURNING t INTO STRICT x;      -- same with
UPDATE ... RETURNING
...etc
```

```
-- e.g.
BEGIN
  SELECT ... INTO STRICT x;

EXCEPTION
  WHEN NO_DATA_FOUND THEN          -- no rows
    ...
  WHEN TOO_MANY_ROWS THEN          -- > 1 rows
    ....
END;
```

==========
======

== Syntax ==

*** BEGIN/END blocks**

Ref: https://www.postgresql.org/docs/current/plpgsql-structure.html

```
---- Blocks
--
-- Basic
DECLARE
  x int;
BEGIN
  RAISE INFO 'x=%', x;
END;

-- DECLARE is optional
BEGIN
  RAISE INFO 'hello';
END;


-- Nesting
DECLARE
  x int = 10;
BEGIN
  -- nest without DECLARE
  BEGIN
    ...
  END;

  -- nest with DECLARE
  DECLARE
   x int = 100;
  BEGIN
    ...
  END;
```

```
  -- RETURN from inner block
  BEGIN
    RETURN;      -- OK

    RETURN 1;    -- ERROR. Can't return
  END;
END;


-- Scope and labels
--
-- Lexical scope
<<o>>                              -- outer scope o
DECLARE
  x int := 1;
BEGIN
  RAISE INFO '% %', x, o.x;        -- x = 1
                                   -- o.x = 1   explicit
block

  <<i>>
  DECLARE
    x int := 10;                   -- case: inner block
  BEGIN
    RAISE INFO '%, %, %',          -- x = 10
shadow
      x, i.x, o.x;                 -- i.x = x = 10
explicit inner block
                                   -- o.x = 1
explicit outer block

    o.x := -1;                     -- OK. Can set outer
block
  END;
  RAISE INFO '%', x;               -- o.x = x = -1
END;
```

*** DECLARE**

Ref: https://www.postgresql.org/docs/current/plpgsql-declarations.html

e.g.

```
-- with declaration
CREATE FUNCTION f() RETURNS INT AS $$
DECLARE
  x INT := 1;
BEGIN
  RETURN x;
END;
$$ LANGUAGE PLPGSQL;
```

```
-- without declaration
CREATE FUNCTION f(x INT) RETURNS INT AS $$
BEGIN
  RETURN x;
END;
$$ LANGUAGE PLPGSQL;
```

```
-- Basic syntax
x int;               -- x is NULL
x int := 1;          -- initialize with 1
x int DEFAULT 1;     -- initial value
x int = 1;           -- same
x int = 1 + 3;       -- any expression
x my_table = (       -- subquery
 SELECT (my_table)   -- GOOD
 SELECT *            -- ERROR
 FROM my_table
 WHERE id=1
);

x int NOT NULL;
x CONSTANT int = 1;
x ALIAS FOR $1;

-- Types
x int;
x y%TYPE;            -- type of variable y e.g.
                     -- e.g. y%type = int  if y is an
int
x mytable%ROWTYPE;   -- type of a row
x RECORD;            -- any ROW type
```

<u>* RETURN values</u>

3 types of returns

1) **RETURN** with **no value**
i.e. procedure
e.g.

```
CREATE PROCEDURE p() AS $$
BEGIN
  RAISE INFO 'hello';
  RETURN;
END;
```

2) **RETURN** with **single row**
i.e. function

3) **RETURN** with **multiple rows**
e.g.

```
create or replace function f() RETURNS SETOF INT AS $$
BEGIN
  RETURN NEXT 1;                                -- append
single row to result
  RETURN QUERY VALUES (10), (20);               -- append
multiple rows to result
  RETURN;                                       -- finish
END;
$$ LANGUAGE PLPGSQL;

SELECT * FROM
 f
────
  1
 10
 20
```

```
-- Syntax
RETURN 1;
RETURN SELECT 1;          -- error
RETURN (SELECT 1);        -- OK subquery
RETURN;                   -- early exit

-- multiple rows
RETURN NEXT 1;                 -- append single
RETURN QUERY (SELECT ...);     -- append multiple from
subquery
RETURN QUERY                   -- dynamic subquery
  EXECUTE 'VALUES (1)';
RETURN QUERY
  EXECUTE 'VALUES ($1), ($2)'
  USING 1,2;
RETURN;
```

*** Basic syntax**
Ref: https://www.postgresql.org/docs/current/plpgsql.html

```
-- Basic
NULL;                -- do nothing
x = 1                -- assign
x := 1

-- qualified names
x
f.x              -- qualified function parameter e.g.
f(x int)
b.x              -- qualified block parameter
```

```
-- e.g. x with various scopes
> CREATE PROCEDURE f(x INT) AS $$      -- fully qualified
f.x
<<b>>
DECLARE
  x INT := 10                          -- fully qualified
b.x
BEGIN
  RAISE INFO '
x=%                                    -- 10
f.x=%                                  -- -1
b.x=%                                  -- 10
', x, f.x, b.x;
END;
$$ LANGUAGE PLPGSQL;


> CALL f(-1);
x=10
f(x)=-1
b.x=10
```

```
-- ASSERT
--
-- Enabled only if
--
-- SET plpgsql.check_asserts='on'
--
ASSERT x = y;
ASSERT x = y, 'FAIL';


---- Conditions
--
-- IFs
IF x = 1 THEN
   ...
ELSIF x = 2 THEN
   ...
ELSE
  ...
END IF;


-- CASE simple
CASE x
  WHEN 1 THEN
    ...
  WHEN 2, 3 THEN      -- multiple cases
   ...
```

```
  ELSE
END CASE


-- CASE general
CASE
  WHEN x = 1 THEN
    ...
  WHEN x = 2 THEN
   ...
  ELSE
END CASE
```

*** Loop**

```
---- Looping
--
--
-- Basics
<<a>>                     -- optional label 'a'
a                         -- ERROR MUST be of the form
<<LABEL-NAME>>
LOOP
  ...
  CONTINUE;
  CONTINUE a;             -- label i.e. continue loop a
  CONTINUE WHEN x > 0;    -- conditional

  EXIT;
  EXIT a;                 -- label
  EXIT WHEN x > 0;        -- conditional
END LOOP;


-- LOOPs
--
-- do LOOP: loop until explicit exit
LOOP
  ...
  EXIT;
END LOOP;

-- WHILE LOOP: loop until condition
WHILE x > 0 LOOP
  ...
END LOOP;
```

```
> DO LANGUAGE PLPGSQL $$
DECLARE
  c int := 0;
BEGIN
  LOOP
    RAISE INFO '%', c;
```

```
    EXIT WHEN c > 10;
    c := c + 1;
  END LOOP;
END;
$$;

INFO:  0
INFO:  1
INFO:  2
INFO:  3
INFO:  4
INFO:  5
INFO:  6
INFO:  7
INFO:  8
INFO:  9
INFO:  10
INFO:  11
```

```
-- FOR i LOOP
--
-- NOTE: i is declared automatically
FOR i IN 10..20 LOOP           -- i = 10,11,...20
FOR i IN 10..20 BY 2 LOOP      -- explicit step e.g. i =
10,12,14...
FOR i IN REVERSE 10..20 LOOP   -- i= 20,18,...10
  ...
END LOOP;

-- FOREACH: loop array
FOREACH c IN ARRAY a LOOP
FOREACH c SLICE 1 IN ARRAY a LOOP          -- SLICE:
dimension of array
                                           -- 0 =
default
  ...
END LOOP;
```

```
DECLARE
  c int := 0;
BEGIN
  -- FOR LOOP
  FOR i in 10..20 by 2 LOOP                -- i declared
automatically
    RAISE INFO '%', i;                     -- i visible
here
  END LOOP;

  -- FOREACH LOOP
  FOREACH c IN ARRAY (ARRAY[1,2,3]) LOOP
    RAISE INFO '%', c;
```

```
  END LOOP;
END;
```

```
-- FOR IN loo
--
-- e.g. loop series
d date;

FOR d IN
  SELECT *
  FROM generate_series('2020-01-06'::date, '2025-01-01',
interval '7 days') _(d)
LOOP
  -- 2020-01-06, 2020-01-13, 2020-01-20, ...
  RAISE INFO '%', d;
END LOOP;
```

==========
======

=== Diagnostics ===

*** RAISE aka logging, error messaging**
Ref: https://www.postgresql.org/docs/current/plpgsql-errors-and-messages.html

RAISE
Command for both **logging** and error throwing
e.g.

```
-- usage 1: logging
RAISE INFO 'x=%', 1                    -- Not an error
 USING HINT='a'


-- usage 2: throw EXCEPTION
RAISE EXCEPTION plgpsql_error          -- throw
   USING MESSAGE='OOPS';

EXCEPTION                              -- catch
  WHEN plpgsql_error THEN
    ...
```

ERRCODE
Ref: https://www.postgresql.org/docs/current/errcodes-appendix.html#ERRCODES-TABLE

P0000	plpgsql_error
P0001	raise_exception
P0002	no_data_found

P0003	too_many_rows
P0004	assert_failure
-	others
MY001	custom code any 5 letter alphanum

e.g.

```
DO LANGUAGE PLPGSQL $$
DECLARE
  msg_state text;
  msg_message text;
  msg_detail text;
  msg_hint text;
  msg_context text;
  msg_schema text;
  msg_table text;
  msg_column text;
  msg_constraint text;
  msg_datatype text;

BEGIN
  RAISE SQLSTATE 'YUI00' USING                  -- Step
1: RAISE EXCEPTION
    MESSAGE='oops',
    DETAIL='some internal error',
    HINT='no hint',
    SCHEMA='myschema',
    TABLE='mytable',
    COLUMN='col1',
    DATATYPE='text',
    CONSTRAINT='CHECK'
    ;

  EXCEPTION                                     -- Step
2: catch EXCEPTION
    WHEN SQLSTATE 'YUI00' THEN
      GET STACKED DIAGNOSTICS                   -- Step
3: DIAGNOSTICS
        msg_state := RETURNED_SQLSTATE,
        msg_message := MESSAGE_TEXT,
        msg_detail := PG_EXCEPTION_DETAIL,
        msg_hint := PG_EXCEPTION_HINT,
        msg_context := PG_EXCEPTION_CONTEXT,
        msg_table := TABLE_NAME,
        msg_schema := SCHEMA_NAME,
        msg_column := COLUMN_NAME,
        msg_constraint := CONSTRAINT_NAME,
        msg_datatype := PG_DATATYPE_NAME
        ;

      RAISE INFO '
state=%
message=%
```

```
context=%
detail=%
hint=%
schema=%
table=%
column=%
constraint=%
data type=%
',
        msg_state,
        msg_message,
        msg_context,
        msg_detail,
        msg_hint,
        msg_schema,
        msg_table,
        msg_column,
        msg_constraint,
        msg_datatype
        ;
END;
$$;
```

Output

```
INFO:
state=YUI00
message=oops
context=PL/pgSQL function testerror() line 14 at RAISE
detail=some internal error
hint=no hint
schema=myschema
table=mytable
column=col1
constraint=CHECK
data type=text
```

```
-- RAISE: Logging
--
-- 3 formats
-- a) simple message format
RAISE INFO 'hello';
RAISE INFO 'hello % %', 1, 'a';           -- template
                                          --  Only % is
supported
RAISE INFO 'hello %s', 1;                 -- BAD Not
format()
RAISE INFO 'hello' || 'a';                -- ERROR
Message cannot be an expression
RAISE INFO format(...)                    -- ERROR
```

```
message := 'a';
RAISE INFO message;                          -- ERROR
RAISE INFO '%', message;                     -- OK


-- b) general message expression
RAISE INFO USING
  MESSAGE='hello'
  MESAAGE='hello' || 'a'                     -- OK any
expression
  MESSAGE=format('hello %s %s, 1, 'a');    -- OK
template


-- levels
RAISE ...        -- default is EXCEPTION
RAISE DEBUG ...
RAISE LOG ...
RAISE INFO ...
RAISE NOTICE ...
RAISE WARNING ...
RAISE EXCEPTION ...


-- OPTIONS: provide additional context
-- See GET STACKED DIAGNOSTICS below
RAISE 'hello' USING
  -- basic context
  DETAIL="abc",
  DETAIL=format("abc"),           -- can use any
expression
  HINT="...",

  -- additional context
  SCHEMA="...",
  TABLE="...",
  COLUMN="...",
  CONSTRAINT="...",
  DATATYPE="...",
```

```
-- e.g.
RAISE EXCEPTION 'OOPS' USING
  DETAIL = 'something is wrong',
  HINT = 'check lin 1',
  SCHEMA = 'publics',
  DATATYPE = 'mytype';

-- output
ERROR:  OOPS
DETAIL:  something is wrong
HINT:  check lin 1
CONTEXT:  PL/pgSQL function testraise() line 7 at RAISE
```

```
-- RAISE: throw EXCEPTION
--
RAISE EXCEPTION division_by_zero;
RAISE EXCEPTION SQLSTATE '22012';          -- same with
code
RAISE division_by_zero;                    -- implicit
EXCEPTION
RAISE EXCEPTION SQLSTATE 'ABC00';          -- Custom
code. Must be 5 letters.
RAISE EXCEPTION SQLSTATE 'ABC000';         -- ERROR Must
be 5 letters.
```

```
-- all same
RAISE EXCEPTION division_by_zero
  USING MESSAGE=format('error %s', 1);

RAISE EXCEPTION SQLSTATE '22012'
  USING MESSAGE=format('error %s', 1);

RASE EXCEPTION 'error %s', 1
  USING ERRCODE='division_by_zero'

RASE EXCEPTION 'error %s', 1
  USING ERRCODE='22012'
```

*** EXCEPTION: catch error**

Ref: https://www.postgresql.org/docs/current/plpgsql-control-structures.html#PLPGSQL-EXCEPTION-DIAGNOSTICS

EXCEPTION

Catches errors thrown between **BEGIN** and **EXCEPTION**.

e.g.

```
DECLARE
  err_msg text;
BEGIN

    ...work ...

EXCEPTION
    -- catches any error between BEGIN and EXCEPTION
   WHEN OTHERS THEN
      GET STACKED DIAGNOSTICS err_msg := MESSAGE_TEXT;
      RAISE INFO 'error: %', err_msg;
END;
```

```
-- EXCEPTION: catch error
--
EXCEPTION
  WHEN no_data_found THEN                          --
with error name
```

```
  WHEN NO_DATA_FOUND THEN                               --
case insensitive
  WHEN SQLSTATE 'P0002' THEN                            --
same with code
  WHEN no_data_found OR too_many_rows THEN              --
multiple
    ...
  WHEN assert_failure THEN                              --
another WHEN
   ...

  WHEN OTHER THEN                                       --
SPECIAL: OTHER catches all errors
   ...


-- STACKED DIAGNOSTICS:
--
-- MUST be inside exception handler
GET STACKED DIAGNOSTICS
  -- info: see RAISE .. USING <options>
  t := RETURNED_SQLSTATE
  t := MESSAGE_TEXT
  t := PG_EXCEPTION_DETAIL
  t := PG_EXCEPTION_HINT
  t := PG_EXCEPTION_CONTEXT
  t := SCHEMA_NAME
  t := TABLE_NAME
  t := COLUMN_NAME
  t := CONSTRAINT_NAME
  t := PG_DATATYPE_NAME
```

==========
======

=== OLD ===

```
This
is
code
Monaco 14
```